2024 2nd International Conference on Networking and Communications (ICNWC)

Computer vision and Generative AI for yield prediction in Digital Agriculture

Sayan Majumder
sm4542@srmist.edu.in

Yash Khandelwal
yk8341@srmist.edu.in

Sornalakshmi K*
sornalak@srmist.edu.in

Department of Data Science and Business Systems,
School of Computing, Faculty of Engineering and Technology,
SRM Institute of Science and Technology, Kattankulathur, Chennai, India 603203
*Corresponding author

2024 2nd International Conference on Networking and Communications (ICNWC) | 979-8-3503-6526-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICNWC60771.2024.10537337

***Abstract*—For the cause of evolution of agriculture to its next generation, the introduction of A.I. and data-driven approach is going to be an important part of the agricultural industry that as per our vision would offer numerous economic, environmental and social benefits. Controlled Environment Agriculture (CEA) is providing more benefits since the weather and other environmental conditions is controlled and predictable. In CEA, the benefits include enhanced productivity in yield, reduced environmental footprints and better resource management. Our solution uses the adoption of Generative AI to visually forecast the potential yield of a crop using stable diffusion and conditional prompts additionally implementing several Generative Adversarial Networks (GAN) architectures for data generation, augmentation and super-resolution. Our proposed system also adopts generative AI for real time monitoring of plants. studying their respective conditions and their autonomous cultivation and harvesting patterns. The dataset used contained over 8479 examples for different fruits alongside annotations. This data was used to train the detection models as well categorically feed the GANs for data generation. Data generated by DCGAN has a structural similarity of 0.5-0.7 whereas data generated by pix2pix network returns us a structural similarity of 0.7-0.9.**

Keywords—CEA: Controlled Environment Agriculture, Generative AI, GAN: Generative Adversarial Network, Stable Diffusion Models

.

I. Introduction

Precision agriculture is gaining popularity because of improving productivity through decision support systems. Computer Vision techniques provide a clear guidance in futuristic decision making or prediction activities. At our earlier stages of using computer vision in agricultural work, we focused on fruit detection and yield counting to draw insights in a solution leveraging productivity for suburban and rural farmers. Our primary objective is to create a strong system which should be able to accurately identify and count end results, greens, and vegetation at various levels of their growth. This would be implemented through state-of-the-art algorithms that meticulously quantify the produce, allowing precise estimation of the harvest. As a benefit of the integration of controlled environment agricultural practices and modern computer vision we would not just enhance accuracy but also streamline the decisionmaking process, fostering more efficient and knowledgeable agricultural practices. With the uprising of Generative AI , we upskilled ourselves and came up with this idea to implement diffusers/Vision Transformers to visually forecast the growth of a crop in a precision agriculture system with good quality data and the help of open source we came up with this solution , with careful finetuning or data centrically modifying a model alongside well-constructed prompts and conditional designing of the system we achieved this milestone. Diffusers along-side conditional GANs could generate a wide variety of possible crop conditions and fruits that could be expected as a yield . The GANs we implement for data-generation are namely DCGAN and pix2pix .

II. Related work

The authors in [1] use deep convolutional neural networks to extract features and analyse the health condition of a plant. In [2], the authors predict the yield of gerbera flower using computer vision techniques. The work in [3] provides an in-depth survey of the artificial techniques that could be used in agriculture. The work also discusses the general flow of image based work in agriculture and also compares the state of the art techniques. The model in [4] recognizes vegetables using the vision transformer model. The paper [5] talks about the evolution of AI and Computer Vision Technologies in Controlled Environment Agriculture Systems. It broadly discusses five key subareas of computer vision and how they related to these CEA problems as researchers a bird-eye view of the striving research are and spark innovation. The authors' work in [4] contains in-depth guide and experiment data on various Vision Transformers available in the current market with wide diversity of architectures and state of the art models. These solutions focus on transfer learning and finetuning concepts as the current revolution of AI has shifted from being model centric to data centric with the rise of Generative AI , LLMs and Vision Transformer. It provides valuable insights for documenting vision-based massive data repositories and feeding them into the existing Vision Transformers easily available within Hugging Face Transformers package. This paper [6] talks about how a U-Net architecture can be used in general computer vision tasks like segmentation or mask generation. Our primary focus in this project is to bring out the next generation of agriculture by incorporating stable diffusers to visually guide agriculturists and researchers about the potential yield from a crop. Diffusers could require a lot of data and finetuning for yielding results with less bias and randomness , to satify that data hungry nature of the same. We have incorporated GANs to generate more and more data close to the ground truth , powered by GPT-3 prompt generation we could potentially achieve a prompt based image translator that can give us the desired results. The generated

979-8-3503-6526-9/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Nitte Meenakshi Institute of Technology. Downloaded on August 13,2025 at 05:19:14 UTC from IEEE Xplore. Restrictions apply.

visual predictions along with our fruit detection models which includes object counting capabilities can help us in developing an analytical system that can boost decision making process.

With current advancement in Data Science, it is not the biggest challenge to build an analytical model from historical data, although it's the best in the business, we are planning to retain the simplicity and effectiveness of such model adding a touch of modernity by incorporating computer vision techniques like the lates object detection models and open source vision transformers from Hugging Face which can do the task of detection and generation with an unconventional with respect to traditional approaches ,transformer-based architecture consisting of encoders and decoders , downsampler – upsamplers , etc., opening a wide variety of tools to explore and play around with the vision to make an impact. Not only detection , by incorporating GANs and stable diffusers as discussed earlier we are here to bring innovation and help fellow researchers with these modern computer vision tools like :- YOLOv8 , YOLO-NAS and Generative AI.

III. Proposed System

Our system consists of various components for the task of detection taking care of traditional model-centric and datacentric approach precisely termed as model finetuning/transfer learning and pruning. Implementation of GANs for image data generation in our work uses transformer-based diffusers to visually predict the growth and yield of a plant. Gathering all the stated components we can expect a system with controlled physical conditions suitable for a specific plant growth where we can visually predict the growth of a plant using diffusers. To generate more data for continuous model improvement we would incorporate the use the GANs for data generation for finetuning diffusers and augmentation strategies specifically for detection models. The detection model will aid us in monitoring and analysing the yield of each crop .

A. Data Generation using DCGAN and pix2pix GAN

For this module we had experimented with DCGAN and pix2pix GAN. The major differences lie in both their architecture and output quality. For our inaugural phase of experimenting with GANs we tried out some basic combinations of generator and discriminator. We fed 100 apple images for the network to learn the patterns of the image pixel intensities which leads learning the shapes and color combinations of the image object. The Deep Convolutional GAN (DCGAN) does not learn the boundary mask of an object instead it uses the absolute difference in adjacent pixel intensities to identify the outline of an object.

We had over 2.7 million trainable parameters for discriminator network on 200 data examples followed by 3.57 million trainable parameters. We will discuss the results in more details in the upcoming sections. The results were underwhelming and requires much refinement. Even after 10,000 epochs the model was unable to fit correct number of pixels within the object mask. From our precious experiments on generating Deep-Fake facial images of humans we succeeded the same task with DC-GAN as given in Fig. 1. For the same experiment we had Human Face Cascade Classifier from OpenCV that made our job much easier. It removed the drawback of basic convolutional GANs which was to not being able to identify the object mask.

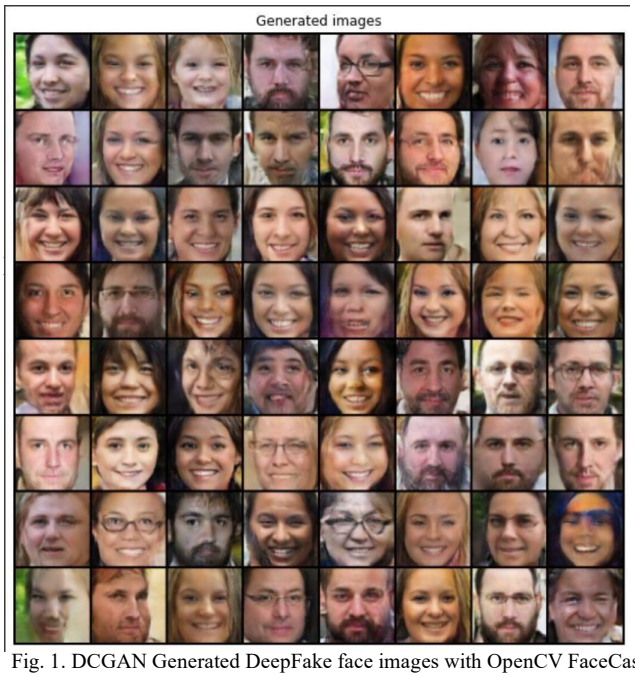

Fig. 1. DCGAN Generated DeepFake face images with OpenCV FaceCascade

Moving onto pix2pix GAN, most success so far. This model is based on a U-Net architecture with up-sampling and down-sampling layers. Here's the step-by-step procedure we implemented to solve this problem:

STEP 1: Preprocess images. Define a transformation pipeline. Our pipeline: i) Convert the images to tensors and send it to the processing device (CPU or GPU, GPU recommended if budget allows). ii) Permute the channels of the image from Height, Width, Channels to Channels, Height, Width. This format is more suitable for working with PyTorch tensors. iii)Take image size as 256 and normalize it to make it within the range of [-1,1].

STEP 2: In this step we will be defining a function to identify the edges of each image and sprinkle them with pixels also known as diffusion but this process is mostly incline towards supervised learning approach.

STEP 3: Split the data into training, test and validation sets.

STEP 4: Define Generator and Discriminator function which allows weight initialization, UNetUp and UNetDown architecture.

The major difference in DCGAN and pix2pix GAN architecture is that DCGAN is a simple GAN with convolutional layers in both Generator and Discriminator , the DCGAN we had implemented had 13 layers in both Generator and Discriminator . It uses convolutions to study the image patters like edges , pixel intensities , etc. Whereas the pix2pix GAN utilizes convolution based U-Net architecture to Downsample images to make it easier to study patterns and Upsamples the generated results to yield better quality output. Pix2pix GAN was able identify the object edges generating outlines or masks , after having learned the pixel intensities

979-8-3503-6526-9/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Nitte Meenakshi Institute of Technology. Downloaded on August 13,2025 at 05:19:14 UTC from IEEE Xplore. Restrictions apply.

and color channels from the training data it was able to diffuse the pixels within the boundary generating some decent quality output as compared to DCGAN , the output quality was much better as DCGANs were incapable of identifying the edges of the objects due to which the pixel diffusion into the background seemed to create a blurry abstract of the actual object.

The data generated using GANs would be used as tool to improvise on the main components of the solution that would be used to feed the diffuser and detection model to counteract potential overfitting and lack of original data in early stages of the development. GANs are not playing any direct role in yield analysis.

Below figure, Fig 2, illustrate our model flow. Figures 3 and 4 show our model performance summary in the generator and discriminator phases.

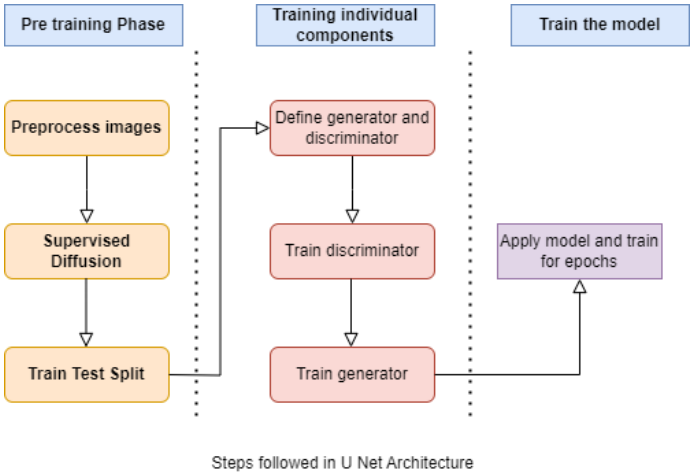

Fig. 2. U Net Architecture

Layer (type)	Output Shape	Param #	Tr. Param #
UNetDown-1	[3, 64, 128, 128]	3,072	3,072
UNetDown-2	[3, 128, 64, 64]	131,072	131,072
UNetDown-3	[3, 256, 32, 32]	524,288	524,288
UNetDown-4	[3, 512, 16, 16]	2,097,152	2,097,152
UNetDown-5	[3, 512, 8, 8]	4,194,304	4,194,304
UNetDown-6	[3, 512, 4, 4]	4,194,304	4,194,304
UNetDown-7	[3, 512, 2, 2]	4,194,304	4,194,304
UNetDown-8	[3, 512, 1, 1]	4,194,304	4,194,304
UNetUp-9	[3, 1024, 2, 2]	4,194,304	4,194,304
UNetUp-10	[3, 1024, 4, 4]	8,388,608	8,388,608
UNetUp-11	[3, 1024, 8, 8]	8,388,608	8,388,608
UNetUp-12	[3, 1024, 16, 16]	8,388,608	8,388,608
UNetUp-13	[3, 512, 32, 32]	4,194,304	4,194,304
UNetUp-14	[3, 256, 64, 64]	1,048,576	1,048,576
UNetUp-15	[3, 128, 128, 128]	262,144	262,144
Upsample-16	[3, 128, 256, 256]	0	0
ZeroPad2d-17	[3, 128, 257, 257]	0	0
Conv2d-18	[3, 3, 256, 256]	6,147	6,147
Tanh-19	[3, 3, 256, 256]	0	0

Total params: 54,404,099
Trainable params: 54,404,099
Non-trainable params: 0

Fig. 3. Generator architecture summary

Layer (type)	Output Shape	Param #	Tr. Param #
Conv2d-1	[3, 64, 128, 128]	6,208	6,208
LeakyReLU-2	[3, 64, 128, 128]	0	0
Conv2d-3	[3, 128, 64, 64]	131,200	131,200
InstanceNorm2d-4	[3, 128, 64, 64]	0	0
LeakyReLU-5	[3, 128, 64, 64]	0	0
Conv2d-6	[3, 256, 32, 32]	524,544	524,544
InstanceNorm2d-7	[3, 256, 32, 32]	0	0
LeakyReLU-8	[3, 256, 32, 32]	0	0
Conv2d-9	[3, 512, 16, 16]	2,097,664	2,097,664
InstanceNorm2d-10	[3, 512, 16, 16]	0	0
LeakyReLU-11	[3, 512, 16, 16]	0	0
ZeroPad2d-12	[3, 512, 17, 17]	0	0
Conv2d-13	[3, 1, 16, 16]	8,192	8,192

Total params: 2,767,808
Trainable params: 2,767,808
Non-trainable params: 0

Fig. 4 Discriminator architecture summary

STEP 5: Define a function to train Generator, Discriminator and optimizer.

STEP 6: Training the discriminator:
i) Discriminator takes source image, real target and fake target as input.
ii) Calculate loss by comparing the real target and the predicted values of the target, where we expect the discriminator to predict the images as real and perform back-propagation. iii) Perform optimizer step and return the overall error and loss values on predicted real and fake target.

STEP 7: Training the Generator: i) Generator takes a fake target and trains it such that it has a low chance of getting identified as fake when passed to the discriminator. ii) For calculating the loss, we calculate pixel loss in addition to generator loss corresponding to difference between the real and generated image of the given contour.

STEP 8: Apply weights and train the model for 100 epochs.

After having mentioned the steps we followed, we explore the significant details of the UNet, Loss functions and Optimization.

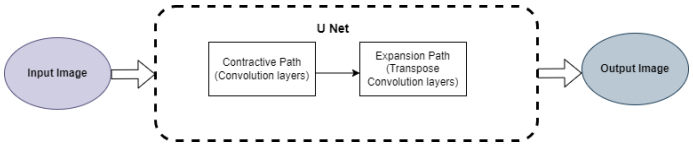

Fig. 5 Components of U Net Architecture

The above Fig. 5 shows what a typical U-Net architecture looks like, it requires two major components:
i) Contractive path: Made up of convolution layers which down-samples the image data while extracting information or pattern recognition. ii) Expensive path: Made up of transpose convolution layer which up-samples the information.

Let's assume our down-sampling has 3 convolutional layers, for such a case we need to make sure that our upsampling has 3 transpose convolution layers. This is because we want to connect the corresponding blocks of the same sizes using skip connection.

979-8-3503-6526-9/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Nitte Meenakshi Institute of Technology. Downloaded on August 13,2025 at 05:19:14 UTC from IEEE Xplore. Restrictions apply.

Down-sampling: This loop consists of several blocks that extract the spatial information, until you reach the middle of that: called the bottleneck, in the last stage. This step signifies the beginning of the upsampling process.

Up-sampling: Therefore, by increasing its data input each subsequent transpose convolutional block produces more to pass on to next from where the information has been concatenated in order to get a safer prediction..

Loss Function:

$$L(G,D) = Ex,y[log\ D(x,y)] + Ex,z[\log\left(D\left(x,G(x,z)\right)\right)]$$

Brief Explanation of the terms in the equation:

- G,D represent Generator and Discriminator
- Ex,y represents the expectation over the joint distribution of real data samples x and y.
    - Log D(x,y): This term measures how discriminator D can distinguish between real data samples (x,y) and fake data samples generated by generator.
    - Ex,z denotes the expectation over the joint distribution of real data samples x and noise vectors z.
    - Log(1-D(x,G(x,z))) : This term measures how well the generator G can fool the discriminator D by making the generated data G(x,z) appear like real data samples(x,y). **Note : D(x,y) = 1 symbolizes real and D(x,G(z)) = 0 is fake.**

The generator in the mixed loss function does two tasks: one is to deceive the discriminator and the other one is to produce images with traits similar to the ground truth. Eventually, the generator's L1 loss was integrated to complement the creational aspect.

LL1(G) = Ex,y,z [|y − G(x,z)|1]

Therefore our final loss function is :

$$G^* = \arg\min_G \max_d L(G,D) + \lambda LL1(G)$$

Loss functions and optimizers go hand-in-hand in any training phase of a model . The entire motive for using 2 loss functions namely pixel loss and generator loss is to do 2 specific tasks :- i) MSE Loss:- Determines how well the Discriminator can classify between fake and real data. ii) L1 Loss:- Determines how well the generator can generate data close to the ground truth.
Optimization:

Our code utilizes Adam optimizers for both generator and discriminator to update their parameters based on a combination of losses. The MSE loss encourages the discriminator to accurately classify real and fake data, while the L1 loss steers the generator towards producing data that closely resembles the real data in terms of pixel values. We used "lambda_pixel" hyperparameter to control the relative importance of these two objectives in the training process.

We would discuss the outcomes in detail in the upcoming sections.

B. Visual forecast of Crop Growth and Yields using Instruction based Image translation

In this module of our work we have incorporated a Stable Diffuser : instructpix2pix . This model is based on pix2pix architecture as discussed in earlier section. In this scenario it has been used for image translation instead of image generation. This conditional stable diffuser model is trained by millions of GPT-3 prompts alongside the corresponding to input. This model has been originally developed by [7]. We are incorporating this model with our gathered/generated image data and prompts to achieve the milestone of visually forecasting the growth of a crop in CEA along with fruit detection and counting modules, introducing a data-driven solution to agriculture industry aiding researchers to expect yield and crop diseases in a more interactive dashboard with yield count expectations .

The results of the models are discussed in the upcoming section.

IV. Results

A. DCGAN Trained on 200 apple images gathered from different Kaggle Datasets

.

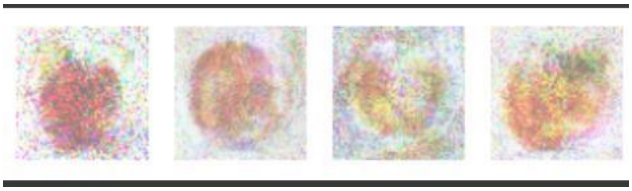

Fig 6 Generated fruit images without cascade

From the above results that we generated in fig.6 we can clearly see that pixel diffusion is irregular and even incomplete, although the model could identify boundaries without pretraining with object contour.

B. Pix2pix GAN: Trained on 200 apple images

Pix2pix GAN demonstrated our experiments in a different level, the U-Net architecture following down-sampling to up-sampling with pre-fed object contours gave us the result we were looking for, even though it's not perfect, we can always improvise.
The sample results is as follows as given in Fig 7.

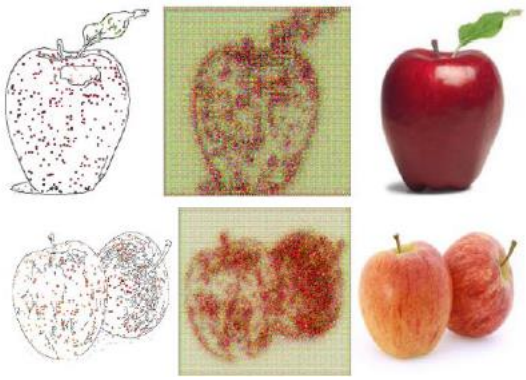

Fig 7 Generated fruit images from U-Net

979-8-3503-6526-9/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Nitte Meenakshi Institute of Technology. Downloaded on August 13,2025 at 05:19:14 UTC from IEEE Xplore. Restrictions apply.

After 100 epochs the model managed to generate images similar to ground truth from the given contours in the left most section of the above image with 90% structural similarity.

Training any computer vision model always requires a budget for the environment based on GPU , CPU and memory consumptions and what device to use (Usually GPU).

Then comes preprocessing the data , as for GANs it is very important to define and implement image transformation to require format , as in this case , tensors and then permute the channels of the matrix to required format then comes normalization to bring the data points within the range [-1,1].

Define model architecture as mentioned followed by a trainer function with required parameters. Then train discriminator alongside generator such that the input data gets processed through discriminator first and the weights for the same is stored and used by generator. We keep doing it iteratively until we reach last epoch. After being done with the training , save and apply the weights to export the trained model and use the same for future use. The major challenges we faced here is while transforming image data that sometimes used to mess up the color channels, secondly the building the loop for training discriminator and generator required attention to minute details of how each networks were interacting with each other.

C. Image Translation using Stable Diffusion

As discussed, we incorporated Timothy Brooke's model known as instruct-pix2pix stable diffuser which a prompt-based image to image diffuser. We implemented this model by adding GPT 3 generated data along with crop data for the model to learn from the respective embedding with ample examples, it can be continually developed for future use.

We have represented below, a sample set of input image in Fig 8, prompt and the output in Fig 9.

Fig 8 Input Image

Input prompt: This tree after 3 months.

Fig 9 Output Image

Instruct-pix2pix was one of the few diffusers which had an open source repository to be developed and used. Upon experimentation we found out that it was already contextually pretrained to our use case but to a very limited amount. Upon uploading the image of a tree , it can identify and visually predict how a similar tree would look like or yield in a span of certain time.

Although it sounds very rewarding to be so close to such a solution, we did find out the model does deviate from reality a little bit and does not have conditional training based on physical factors and sometimes the generated result seems a bit random. To fully launch this as a product is still out of scope but with time and training it with more image examples along with some appropriate prompts should lead to a generalised understanding for the models and in future.

Our primary focus would be to launch this system , gather more ground truth data and design the appropriate prompts. Since the model requires an image and a prompt to generate the results. The typical format to follow would be use image data with required prompt and resultant image to train.

For example : We take the image of a plant + "This plant after 3 months"(prompt) -> We take the image of the same plant after 3 months as a result. Training the model on this data format will soon start to show results.

The current system consists of randomness in terms of image translation by diffusers and could potentially be improved by designing the data and prompts to further finetune it as discussed earlier. So far, the detection models haven't shown any weaknesses but larger detection models can be incorporated with more variation and volumes of data.

In the future as we get hold of more and more data with addition to more ground truth data the model would require to have more trainable parameters. That is where the larger detection models come into action with large number of layers or a massive architecture. It woud be the appropriate moment to play around with the model-centric approaches or pruning as discussed earlier. Hence, as the system grows, the scope of experimentation

979-8-3503-6526-9/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Nitte Meenakshi Institute of Technology. Downloaded on August 13,2025 at 05:19:14 UTC from IEEE Xplore. Restrictions apply.

grows as well. Currently lack of data and randomess of image generation is the issue which is a matter of time.

V. CONCLUSION AND FUTURE WORK

Our conclusion is that the model hasn't been trained to be factual or true representation of physical events yet. Therefore, using this model before further conditional refinement is out of scope. The model does not achieve photorealism and autoencoding part of the model is lossy. Although the capabilities of image generation is impressive it also does reinforce some social or physical biases and the image generation sometimes seem very random and loses the base concept of the image.

References:

[1] P. (Paul M.) Cunningham, M. Cunningham, Institute of Electrical and Electronics Engineers, and IEEE Society on Social Implications of Technology, *2020 IST-Africa Conference : 18-22 May 2020, Virtual Conference*.

[2] M. K. Mishra, Institute of Electrical and Electronics Engineers., Computer Society of India., and GLA University. Department of Computer Engineering & Applications., *Proceedings of the 2013 International Conference on Information Systems and Computer Networks (ISCON) : March 09 -10, 2013, Mathura, India*. IEEE, 2013.

[3] M. T. Habib, D. M. Raza, M. M. Islam, D. B. Victor, and M. A. I. Arif, "Applications of Computer Vision and Machine Learning in Agriculture: A State-of-the-Art Glimpse," in *2022 International Conference on Innovative Trends in Information Technology, ICITIIT 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/ICITIIT54346.2022.9744150.

[4] L.-H. Li and R. Tanone, "Vision Transformer Approach for Vegetables Recognition," in *2022 International Seminar on Application for Technology of Information and Communication (iSemantic)*, 2022, pp. 113–118. doi: 10.1109/iSemantic55962.2022.9920435.

[5] L. U. O. Jiayun, L. I. Boyang, and C. Leung, "A Survey of Computer Vision Technologies in Urban and Controlled-environment Agriculture," *ACM Comput Surv*, vol. 56, no. 5, Nov. 2023, doi: 10.1145/3626186.

[6] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Springer Verlag, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.

[7] T. Brooks, A. Holynski, and A. A. Efros, "InstructPix2Pix: Learning to Follow Image Editing Instructions," Nov. 2022, [Online]. Available: http://arxiv.org/abs/2211.09800

979-8-3503-6526-9/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Nitte Meenakshi Institute of Technology. Downloaded on August 13,2025 at 05:19:14 UTC from IEEE Xplore. Restrictions apply.